

Learning Objectives

Learners will be able to...

- Use automatic taggers such as the `RegexpTagger()`.
- Utilize dictionaries with NLP, with processes such as POS tagging and frequency distributions.
- Define the various classes of n-gram taggers and describe when to use each.
- Store taggers and understand performance limitations with n-gram tagging.

info

Make Sure to Know

Intermediate Python and regex.

Limitations

In this assignment, we only work with the NLTK library.

Apply Automatic Taggers

Use RegexpTagger

The RegexpTagger can be used as a useful tagger for identifying tags of unknown words.

Import the applicable processes from the NLTK library used for applying NLTK's RegexpTagger, UnigramTagger, and DefaultTagger.

For this exercise, we will work with text from the Brown Corpus. Import this corpus as well.

```
from nltk import DefaultTagger, UnigramTagger, RegexpTagger
from nltk.corpus import brown
```

We will store all sentences from the Brown Corpus, reviews category in brown_sentences.

```
brown_sentences = brown.sents(categories='reviews')
```

We create patterns that contains multiple regex patterns identifying various parts of speech tags.

- We defined tag 'NN' as our default tag, for when a word doesn't match another pattern in patterns.

```
patterns = [(r'.*ed$', 'VBD'), ## <- past-tense verb
             (r'.*ing$', 'VBG'), ## <- gerund verb
             (r'.*es$', 'VBZ'), ## <- present-tense, 3rd person, singular verb
             (r'.*ould$', 'MD'), ## <- modal word
             (r'.*\'s$', 'NN$'), ## <- possessive noun
             (r'.*s$', 'NNS'), ## <- plural noun
             (r'^-?[0-9]+(.[0-9]+)?$', 'CD'), ## <- cardinal number
             (r'.*', 'NN')] ## <- noun (default tag)
```

In the code below, we apply the RegexpTagger that we imported earlier from the nltk module against patterns to build a tagger called regexp_tagger. Then, we apply the tagger using the tag method to brown_sentences as created earlier.

Enter the below code, then select the TRY IT button to run the file.

```
regexp_tagger = RegexpTagger(patterns)
print("\nTagged Words in brown_sentences using RegexpTagger:\n",
      regexp_tagger.tag(brown_sentences[37]))
```

Your output should display as shown below.

```
Tagged Words in brown_sentences using RegexpTagger:
[('We', 'NN'), ('printed', 'VBD'), ('it', 'NN'), ('.', 'NN')]
```

Notice how most of the words are tagged as NN. This is because these words do not match any of the previously defined patterns. Therefore they default to NN even if the word is not a noun. Only one word (printed) matched a pattern (past-tense verb).

challenge

Try this variation:

Add a tuple to the patterns variable that checks for the characters ., ,, !, ?, and - and labels them as PNC for punctuation. Then select the TRY IT button to run the file and check your work.

The expected output is shown below.

```
Tagged Words in brown_sentences using RegexpTagger:
[('We', 'NN'), ('printed', 'VBD'), ('it', 'NN'), ('.',
'PNC')]
```

▼ Solution

One possible solution is shown below.

```
patterns = [(r'.*ed$', 'VBD'), ## <- past-tense verb
            (r'.*ing$', 'VBG'), ## <- gerund verb
            (r'.*es$', 'VBZ'), ## <- present-tense, 3rd person, singular
                               verb
            (r'.*ould$', 'MD'), ## <- modal word
            (r'.*\'s$', 'NN$'), ## <- possessive noun
            (r'.*s$', 'NNS'), ## <- plural noun
            (r'^-?[0-9]+(\.[0-9]+)?$', 'CD'), ## <- cardinal number
            (r'[.,!?-]', 'PNC'), ## <- punctuation
            (r'.*', 'NN')] ## <- noun (default tag)
```

Use DefaultTagger

In this next exercise, we will continue to work with the Brown Corpus. Additionally, we will work with frequency distributions and a `word_tokenize`.

```
from nltk.probability import FreqDist
from nltk.tokenize import word_tokenize
```

Create list tags that contains each word and POS tag from the reviews category of the tagged_words in the Brown Corpus. Use `FreqDist` along with the `max` function to tag the most common, or most likely, POS tags in the text.

```
tags = [tag for (word, tag) in
          brown.tagged_words(categories='reviews')]
print("\nThe most common / likely tag of each word in
      brown_sentences:", FreqDist(tags).max()) ## the tag that
      is most likely: 'NN'
```

Your output should display as shown below.

```
The most common / likely tag of each word in brown_sentences: NN
```

Create string `blankets` and `blanket_tokens`. Using `DefaultTagger`, set the default tag for words in text to 'NN', and store this tagger in `default_tagger`.

Finally, apply `default_tagger` to `blanket_tokens` as shown in the code below.

- **Note:** All words should be tagged as 'NN', since this is set as the default tag.

```
blankets = 'Blankets should be warm and soft, but not hot or
           light.'
blanket_tokens = word_tokenize(blankets)
default_tagger = DefaultTagger('NN')
print("\nTagged Words in blankets using DefaultTagger:",
      default_tagger.tag(blanket_tokens)) ## all words will be
      tagged with 'NN' as a default, which results in a very
      poor accuracy percentage
```

Your output should display as shown below.

```
Tagged Words in blankets using DefaultTagger: [('Blankets',  
'NN'), ('should', 'NN'), ('be', 'NN'), ('warm', 'NN'), ('and',  
'NN'), ('soft', 'NN'), (',', 'NN'), ('but', 'NN'), ('not',  
'NN'), ('hot', 'NN'), ('or', 'NN'), ('light', 'NN'), ('.',  
'NN')]
```

Utilize Dictionaries with NLP

Use defaultdict

▼ Refresher - dictionaries

Create an empty dictionary `pos_tags1`, then manually populate it with one key-value pair as shown below.

```
pos_tags1 = {}  
pos_tags1['entertain'] = 'V'  
print("Dictionary, manually populated:", pos_tags1)
```

So far, `pos_tags1` contains one key-value pair: `{'entertain': 'V'}`, where 'entertain' is the key and 'V' is a value.

We can retrieve values by using keys as shown below. Note that we cannot retrieve values by using keys that haven't been assigned values.

```
print(pos_tags1['entertain'])
```

If we want to only retrieve keys from the dictionary, we can..

1. Convert the dictionary to a list.

```
python-hide-clipboard    print(list(pos_tags1))
```

1. Use the built-in dictionary method `keys` to access keys.

```
python-hide-clipboard    print(pos_tags1.keys())  
print(list(pos_tags1.keys()))
```

Other Useful Built-In Dictionary Functions:

- `values`: used to retrieve the values in a dictionary.

- `items`: used to retrieve the key-value pairs in a dictionary.

We can also define dictionaries using either format as shown below with dictionary `pos_tags2`.

1. Use curly braces `{}`.

```
python-hide-clipboard    pos_tags2 = {'entertain': 'V', 'lane':  
'N', 'vain': 'ADJ', 'plain': 'ADJ'}
```

1. Use the dict constructor.

```
python-hide-clipboard    pos_tags2 = dict(entertain= 'V', lane=  
'N', vain= 'ADJ', plain= 'ADJ')
```

Like any other data type (such as a dictionary), `defaultdict` is a container. With nearly the same functionality as a dictionary, `defaultdict` does not raise `KeyErrors`, but rather produces a default value for a specified key that does not exist.

- It is useful to work with `defaultdict` when trying to access a specified key not yet in a dictionary.

Import the applicable processes from the `collections` module used for applying `defaultdict`.

```
from collections import defaultdict
```

For this exercise, we will work with dictionary `pos_tags`, as shown below.

```
pos_tags = {'education': 'N', 'neat': 'ADJ', 'extend': 'V',  
           'sweet': 'ADJ', 'deflation': 'N', 'bend': 'V'}
```

Predict what you think will happen when we attempt to print out the POS tag for both the strings 'bend' and 'dog' in dictionary `pos_tags`, as shown in the code below.

Enter the code, then select the TRY IT button to run the file.

```
print("Original POS Tag(s) for 'bend' before using  
      defaultdict:", pos_tags['bend'])  
print("Original POS Tag(s) for 'dog' before using defaultdict:",  
      pos_tags['dog'])
```

Your output should display as shown below.

```
Traceback (most recent call last):  
print("Original POS Tag(s) for 'dog' before using defaultdict:",  
pos_tags['dog'])  
KeyError: 'dog'
```



What does the error mean?

```
Traceback (most recent call last):  
print("Original POS Tag(s) for 'dog' before using defaultdict:",  
pos_tags['dog'])  
KeyError: 'dog'
```

Because we are trying to access key 'dog' that is not currently contained in dictionary pos_tags, the computer throws a KeyError.

This is why it is useful to work with defaultdict when trying to access a specified key not yet in a dictionary without receiving an error.

debugging

Important: Before moving on, make sure to comment out or delete the following lines from your code.

```
print("Original POS Tag(s) for 'dog' before using defaultdict:", pos_tags['dog'])
```

Otherwise, the computer will get stuck on this error when running the file again on this page.

In order to apply defaultdict, we must specify the default data type value as a parameter to defaultdict.

Default values, such as int, float, str, list, dict, tuple, and so forth are functions that convert objects to the specified data type.



Explained - default value functions

When the object exists, the function will take the object as a parameter and convert it to the specified data type. Two examples are shown below.

```
- int('2') -> 2.  
- list('2') -> ['2']
```

When calling a default value function on an object that doesn't exist (the parameter is empty), the computer will insert the default value for the specified data type. Two examples are shown below.

```
- int() -> 0  
- list() -> []
```


In the code shown below, we set the parameter to list in defaultdict for variable pos_tags_list1.

- We create a list to represent the possible POS tags associated with string 'bend'. Then, we attempt to return this list.
- We attempt to return a list of the possible POS tags associated with string 'dog', which we have not defined.

Predict what you think will happen, enter the code below, then select the TRY IT button to run the file.

```
pos_tags_list1 = defaultdict(list)
pos_tags_list1['bend'] = ['V', 'N']
print("\nList of Possible POS Tags for 'bend' after applying defaultdict(list):",
      pos_tags_list1['bend'])
print("List of Possible POS Tags for 'dog' after applying defaultdict(list):",
      pos_tags_list1['dog'])
```

Your output should display as shown below.

```
List of Possible POS Tags for 'bend' after applying defaultdict(list): ['V', 'N']
List of Possible POS Tags for 'dog' after applying defaultdict(list): []
```

In the code shown below, the parameter to defaultdict is applied using a lambda function for variable pos_tags_list2.

- We create a list to represent the possible POS tags associated with string 'bend'. Then, we attempt to return this list.
- We attempt to return a list of the possible POS tags associated with string 'dog', which we have not defined.
- Lastly, we print a list of all pairs in pos_tags_list2.

Enter the code below, then select the TRY IT button to run the file.

```
pos_tags_list2 = defaultdict(lambda: 'N')
pos_tags_list2['bend'] = ['V', 'N']
print("\nList of Possible POS Tags for 'bend' after applying defaultdict(lambda: 'N'):",
      pos_tags_list2['bend'])
print("List of Possible POS Tags for 'dog' after applying defaultdict(lambda: 'N'):",
      pos_tags_list2['dog'])
print("List of All Items in pos_tags after applying defaultdict(lambda: 'N'):",
      list(pos_tags_list2.items()))
```

Your output should display as shown below.

```
List of Possible POS Tags for 'bend' after applying defaultdict(lambda: 'N'): ['V', 'N']
List of Possible POS Tags for 'dog' after applying defaultdict(lambda: 'N'): N
List of All Items in pos_tags after applying defaultdict(lambda: 'N'): [('bend', ['V',
'N']), ('dog', 'N')]
```

challenge

***Variation A* - Try this variation:**

We can use the update method to combine dictionaries pos_tags and pos_tags2. Notice how both dictionaries have multi-valued keys. Change the code in the IDE to the sample below.

Predict what you think will happen, then select the TRY IT button to run the file.

```
from collections import defaultdict

pos_tags = {'education': 'N', 'neat': 'ADJ', 'extend': 'V', 'sweet': 'ADJ',
            'deflation': 'N', 'bend': 'V'}
pos_tags.update({'computer': 'N', 'read': 'V', 'diligently': 'ADV', 'tired': 'ADJ',
                'slowly': 'ADV', 'walk': 'V'})
pos_tags2 = defaultdict(list)
for key,value in pos_tags.items():
    pos_tags2[value].append(key)

print("\nOriginal pos_tags dictionary combined with new pos_tags2 dictionary:",
      pos_tags2)
```

The expected output is shown below.

```
Original pos_tags dictionary combined with new pos_tags2 dictionary: defaultdict(<class
'list'>, {'N': ['education', 'deflation', 'computer'], 'ADJ': ['neat', 'sweet',
'tired'], 'V': ['extend', 'bend', 'read', 'walk'], 'ADV': ['diligently',
'slowly']})
```

As we can see, the update method allows us to combine dictionaries with multi-valued keys. As an example, print all values paired with key 'V' as shown below.

Enter the code, then select the TRY IT button to run the file.

```
print("All words in pos_tags_2 that are identified as 'V':", pos_tags2['V'])
```

The expected output is shown below.

```
All words in pos_tags_2 that are identified as 'V': ['extend', 'bend', 'read', 'walk']
```

challenge

***Variation B* - Try this variation:**

Instead of using the update method as we did in *Variation A* above, we can alternatively use NLTK's Index module to combine dictionaries with multi-valued keys, such as pos_tags and pos_tags3. Change the code in the IDE to the sample below.

Predict what you think will happen, then select the TRY IT button to run the file.

```
from nltk import Index

pos_tags = {'education': 'N', 'neat': 'ADJ', 'extend': 'V', 'sweet': 'ADJ',
            'deflation': 'N', 'bend': 'V'}

pos_tags3 = Index((value, key) for (key, value) in pos_tags.items())
print("\nOriginal pos_tags dictionary combined with new pos_tags3 dictionary:",
      pos_tags3)
print("All words in pos_tags_3 that are identified as 'V':", pos_tags3['V'])
```

The expected output is shown below.

```
Original pos_tags dictionary combined with new pos_tags3 dictionary: Index(<class
'list'>, {'N': ['education', 'deflation'], 'ADJ': ['neat', 'sweet'], 'V':
['extend', 'bend']})
All words in pos_tags_3 that are identified as 'V': ['extend', 'bend']
```

Map Frequent Words & Hapaxes

We can apply defaultdict in order to replace all low-frequency words with the string 'unknown word'.

For this exercise, we will work with the Gutenberg Corpus. We will also be utilizing frequency distributions and POS tags. Import these modules as shown below.

```
from nltk.corpus import gutenberg
from nltk.probability import FreqDist
from nltk import pos_tag
```

In the code below, we store a list of words in variable `macbeth` as taken from the text file called 'shakespeare-macbeth.txt' from the Gutenberg Corpus.

- We store the frequency of each word listed in `macbeth` in variable `macbeth_words_freqDist` using `FreqDist`.
- We create a list called `macbeth_words_mostCommon1000`, which stores the most common 1000 words in `macbeth_words_freqDist`.

```
macbeth = gutenberg.words('shakespeare-macbeth.txt')
macbeth_words_freqDist = FreqDist(macbeth)
macbeth_words_mostCommon1000 = [word for (word, _) in
                                macbeth_words_freqDist.most_common(1000)]
```

In the next chunk of code below, the parameter to `defaultdict` is applied using a `lambda` function and is stored in variable `macbeth_words_map`.

- Using a `for` loop, we iterate through each word listed in `macbeth_words_mostCommon1000` to apply `macbeth_words_map` to each word. This will map each of the most common 1000 words to itself.
- We create list `macbeth2` that stores the value of each word in `macbeth` after applying each word to `macbeth_words_map`. This will map each word in `macbeth` to its replacement: 'unknown word'.

```
macbeth_words_map = defaultdict(lambda: 'unknown word')
for word in macbeth_words_mostCommon1000: macbeth_words_map[word] = word

macbeth2 = [macbeth_words_map[word] for word in macbeth]
```

Enter the code below, then select the TRY IT button to run the file.

```
print("\nList of First 20 Words in Macbeth with all low-frequency words marked as 'unknown word':", macbeth2[:20])
print("List of First 20 Words in Macbeth with Tagged POS (minus low-frequency words):", pos_tag(macbeth2[:20]))
```

The expected output is shown below.

```
List of First 20 Words in Macbeth with all low-frequency words marked as 'unknown word':  
['[', 'The', 'unknown word', 'of', 'Macbeth', 'by', 'unknown word', 'unknown word',  
'unknown word', ']', 'Actus', 'unknown word', '.', 'unknown word', 'Prima', '.', 'Thunder',  
'and', 'Lightning', '.']  
List of First 20 Words in Macbeth with Tagged POS (minus low-frequency words): [('[',  
'IN'), ('The', 'DT'), ('unknown word', 'NN'), ('of', 'IN'), ('Macbeth', 'NNP'), ('by',  
'IN'), ('unknown word', 'JJ'), ('unknown word', 'NN'), ('unknown word', 'NN'), (']',  
'NNP'), ('Actus', 'NNP'), ('unknown word', 'NN'), ('.', '.'), ('unknown word', 'JJ'),  
('Prima', 'NNP'), ('.', '.'), ('Thunder', 'NNP'), ('and', 'CC'), ('Lightning', 'NNP'),  
('.', '.')]
```

challenge

Try this variation:

Find the number of times 'unknown word' is listed as a token in `macbeth2`.

Write the code to produce the following expected output, then select the TRY IT button to run the file and check your work.

The expected output is shown below.

```
Number of Times 'unknown word' Appeared in macbeth2: 3534
```

▼ Solution

One possible solution is shown below, where we can...

- Create a defaultdict variable called `unknown_word_count` that uses the `int` function as its parameter.
- For each sub tuple (word, tag) created to be contained in the list initiated by applying the `pos_tag` function to `macbeth2`, where there is a word present, we will add a counter value of 1 to list `unknown_word_count`.
- Lastly, we print the number of times string 'unknown word' appears by using `unknown_word_count` count all occurrences of element 'unknown word'.

```
unknown_word_count = defaultdict(int)  
for (word, tag) in pos_tag(macbeth2):  
    unknown_word_count[word] += 1  
  
print("\nNumber of Times 'unknown word' Appeared in macbeth2:", unknown_word_count['unknown word'])
```

Use N-Gram Tagging

Definition: N-Gram Tagging

- N-gram taggers tag POS by analyzing the previous words along with their POS tags and by using a context dictionary.
- There are three sub-classes of n-gram taggers, which are the:
 - Unigram Tagger,
 - Bigram Tagger, and the
 - Trigram Tagger.
- The Bigram Tagger uses the first previous word's POS as part of its context, while the Trigram Tagger uses the first two previous word's POS as part of its context.
- **Why is it called n-gram tagging?** That's because n-gram tagging works by using the previous $n-1$ previous words to predict the next word's POS.
 - The unigram tagger is a 1-gram tagger, and likewise, the bigram and trigram tagger are 2- and 3-gram taggers respectively.

Use the UnigramTagger

Within the NLTK library, the NgramTagger class is used for n-gram tagging operations.

- The NgramTagger class uses a tagged training corpus to identify its predicted POS tag based on each context.

Import the applicable processes from the NLTK library used for using n-gram tagging in these exercises. We will use the Brown Corpus for this page.

```
from nltk import UnigramTagger, BigramTagger, DefaultTagger,
    TrigramTagger
from nltk.corpus import brown
```

For this exercise, we will use:

- Variable brown_tagged_sents to represent a list of each word and its tagged POS according to the tagged_sents reader function from the Brown Corpus's News category. And
- Variable brown_sents to represent a list of each word according to the sent_sents reader function from the Brown Corpus's News category.

```
brown_tagged_sents = brown.tagged_sents(categories='news')
brown_sents = brown.sents(categories='news')
```

We define a third variable called `brown_unigram_tagger` to represent a unigram tagger using `UnigramTagger` and variable `brown_tagged_sents`.

- This essentially trains the unigram tagger to tag POS in sentences from the Brown Corpus's News category.

```
brown_unigram_tagger = UnigramTagger(brown_tagged_sents)
```

Finally, we can apply the `brown_unigram_tagger` to the variable `brown_sents`. For this example, we will look at news from 2009.

Enter the below code, then select the TRY IT button to run the file.

```
print(brown_unigram_tagger.tag(brown_sents[2009]))
```

Your output should display as shown below.

```
[('The', 'AT'), ('structures', 'NNS'), ('housing', 'VBG'),
 ('the', 'AT'), ('apartments', 'NNS'), ('are', 'BER'), ('of',
 'IN'), ('masonry', 'NN'), ('and', 'CC'), ('frame', 'NN'),
 ('construction', 'NN'), ('.', '.')]

```


challenge

Try this variation:

Use the `evaluate` function to evaluate the correctness of the output from the Unigram Tagger you just ran called `brown_unigram_tagger` compared to that of the output when using the reader function `tagged_sents` in the variable `brown_tagged_sents`.

Enter the below code, then select the TRY IT button to run the file.

```
print(brown_unigram_tagger.accuracy(brown_tagged_sents))
```

Your output should display as shown below.

```
0.9349006503968017
```

This means that the output from the `UnigramTagger` we created was approximately 93.49% the same as the output from the `tagged_sents`; that is, our Unigram Tagger was approximately 93.49% accurate.

Separate & Store Data

In the above example of unigram tagging, we train the `UnigramTagger` called `brown_unigram_tagger` by feeding it variable `brown_tagged_sents` which is the stored output from applying the reader function `tagged_sents` to the Brown Corpus's News category.

In the variation above, we evaluated `brown_unigram_tagger` by comparing it to the output from applying the reader function `tagged_sents` to the Brown Corpus's News category, and we saw that `brown_unigram_tagger` came out to be approximately 93.49% accurate.

- We trained and evaluated `brown_unigram_tagger` using the same exact data, which didn't produce an entirely accurate result.

By separating our data into a group for training and another group for evaluating, AKA *testing*, we can get a more accurate idea of how useful `brown_unigram_tagger` will be on unseen text.

First, we need to determine the size of the data stored in `brown_tagged_sents`.

- Using the `len` function, we can find the number of total items in the list.

Multiplying the total number of items in list `brown_tagged_sents` by `0.9` will allow us to find how many items are in the first 90% of `brown_tagged_sents`.

- We will be separating data from `brown_tagged_sents` into a group for training and a group for testing. For this exercise, we will use 90% of our data for training.

```
brown_tagged_sents_size = int(len(brown_tagged_sents) * 0.9)
```

Let's separate the data stored in `brown_tagged_sents` into two variables:

- `training_brown_tagged_sents`, which represents 90% of the data, and
- `testing_brown_tagged_sents`, which represents the remaining 10% of the data.

```
training_brown_tagged_sents =  
    brown_tagged_sents[:brown_tagged_sents_size]  
testing_brown_tagged_sents =  
    brown_tagged_sents[brown_tagged_sents_size:]
```

Now we are ready to create our new `UnigramTagger`, which we will call `brown_unigram_tagger2` which is trained using `training_brown_tagged_sents`.

```
brown_unigram_tagger2 =  
    UnigramTagger(training_brown_tagged_sents)
```

Finally, let's evaluate the correctness of `brown_unigram_tagger2` compared to the data in untouched variable `testing_brown_tagged_sents`.

Enter the below code, then select the TRY IT button to run the file.

```
print(brown_unigram_tagger2.accuracy(testing_brown_tagged_sents)  
)
```

Your output should display as shown below.

```
0.8121200039868434
```

As we can see, the value of our correctness is less than before. However, this time, we more fairly evaluated the tagger.

Use N-Gram Tagging

In this next exercise, we will continue to work with variables `training_brown_tagged_sents` and `testing_brown_tagged_sents` to train and test our Bigram Tagger.

We create `brown_bigram_tagger` to represent the `BigramTagger` trained with the data from `training_brown_tagged_sents`.

Print the results of using `brown_bigram_tagger` on the same data as we used before, and evaluate its correctness compared to `testing_brown_tagged_sents`.

Enter the below code, then select the TRY IT button to run the file.

```
brown_bigram_tagger = BigramTagger(training_brown_tagged_sents)
print(brown_bigram_tagger.tag(brown_sents[2009]))
print(brown_bigram_tagger.accuracy(testing_brown_tagged_sents))
```

Your output should display as shown below.

```
[('The', 'AT'), ('structures', 'NNS'), ('housing', 'VBG'),
 ('the', 'AT'), ('apartments', 'NNS'), ('are', 'BER'), ('of',
 'IN'), ('masonry', 'NN'), ('and', 'CC'), ('frame', 'NN'),
 ('construction', 'NN'), ('.', '.')]
0.10206319146815508
```

Now, let's test `brown_bigram_tagger` on a new set of data from the Brown Corpus's News category. We will print the results and also evaluate its correctness compared to `testing_brown_tagged_sents`.

Enter the below code, then select the TRY IT button to run the file.

```
brown_additional_sentences = brown_sents[4205]
print(brown_bigram_tagger.tag(brown_additional_sentences))
print(brown_bigram_tagger.accuracy(testing_brown_tagged_sents))
```

Your output should display as shown below.

```
[('The', 'AT'), ('climate', 'NN'), ('ranges', None), ('from',
None), ('the', None), ('steamily', None), ('equatorial', None),
 ('to', None), ('the', None), ('temperate', None), ('.', None)]
0.10206319146815508
```



Why wasn't the Bigram Tagger more correct?

As n increases so does the contextual specificity of the tagger. The more specific context is used to solely train the tagger, the less accurate it will be when it has to predict the tag of unseen

context.

This is called the *sparse data* problem.

challenge

Try this variation:

We can combine taggers to help address inaccuracies associated with the *sparse data* problem.

- Using the most accurate algorithm for each tag will help improve accuracy and coverage.

First, we define a DefaultTagger called `default_tagger`, which when applied, will tag all words with 'NN'.

We then define three more taggers.

- `tagger1` is a UnigramTagger trained to identify tags based on the data stored in `training_brown_tagged_sents`. However, if this tagger is unable to find a tag for a word, it will run `default_tagger` against the word as a backup tagger.

- Likewise, - `tagger2` is a BigramTagger trained to identify tags based on the data stored in `training_brown_tagged_sents`. However, if this tagger is unable to find a tag for a word, it will run `tagger1` against the word as a backup tagger.

- Likewise, - `tagger3` is a TrigramTagger trained to identify tags based on the data stored in `training_brown_tagged_sents`. However, if this tagger is unable to find a tag for a word, it will run `tagger2` against the word as a backup tagger.

Enter the below code, then select the TRY IT button to run the file.

```
default_tagger = DefaultTagger('NN')
tagger1 = UnigramTagger(training_brown_tagged_sents, backoff =
    default_tagger)
tagger2 = BigramTagger(training_brown_tagged_sents, backoff = tagger1)
tagger3 = TrigramTagger(training_brown_tagged_sents, backoff = tagger2)

print(tagger3.accuracy(testing_brown_tagged_sents))
```

Your output should display as shown below.

```
0.843317053722715
```

As we can see, this is our most fair evaluation of the correctness of a tagger, as it has the most scope. It is also the most accurate of all the taggers, besides - of course - the first tagger we created called `brown_unigram_tagger` which evaluated its correctness based on the same data it used for training.

Store & Analyze Taggers

Create Taggers

Import the applicable processes from the NLTK library used for using n-gram tagging, storing and analyzing tagging in these exercises. We will use the Brown Corpus for this page.

```
from nltk import UnigramTagger, BigramTagger, DefaultTagger,
                 TrigramTagger, ConditionalFreqDist, trigrams,
                 ConfusionMatrix

from nltk.corpus import brown

from pickle import dump, load
```

Prompt 1: Utilizing your experience from the previous page in this assignment titled *Use N-Gram Tagging*, create two new variables:

- brown_sents to represent all sentences in the Brown Corpus's *Reviews* category, and
- tagged_sents to represent all tagged POS for all sentences in the Brown Corpus's *Reviews* category.
- Print the first element in tagged_sents.

Select the TRY IT button when you are ready to run your code.

Your output should display as shown below.

```
[('It', 'PPS'), ('is', 'BEZ'), ('not', '*'), ('news', 'NN'),
 ('that', 'CS'), ('Nathan', 'NP'), ('Milstein', 'NP'), ('is',
 'BEZ'), ('a', 'AT'), ('wizard', 'NN'), ('of', 'IN'), ('the',
 'AT'), ('violin', 'NN'), ('.', '.')] ]
```

▼ Solution

The solution is shown below.

```
brown_sents = brown.sents(categories='reviews')
tagged_sents = brown.tagged_sents(categories='reviews')
print(tagged_sents[0])
```

Prompt 2: Utilizing your experience from the previous page in this assignment titled *Use N-Gram Tagging*, create three new variables:

- tagged_sents_size to represent the number of sentences in 90% of the data stored in tagged_sents.
- training_tagged_sents to represent the first 90% of the sentences stored in tagged_sents.
- testing_tagged_sents to represent the remaining 10% of the sentences stored in tagged_sents.
- Print the first element in testing_tagged_sents.

Select the TRY IT button when you are ready to run your code.

Your output should display as shown below.

```
[('Perhaps', 'RB'), ('he', 'PPS'), ('has', 'HVZ'), ('only', 'RB'), ('forgotten', 'VBN'), ('a', 'AT'), ('great', 'JJ'), ('deal', 'NN'), ('.', '.')] ]
```

▼ Solution

The solution is shown below.

```
tagged_sents_size = int(len(tagged_sents) * 0.9)
training_tagged_sents = tagged_sents[:tagged_sents_size]
testing_tagged_sents = tagged_sents[tagged_sents_size:]
print(testing_tagged_sents[0])
```

Prompt 3: Utilizing your experience from the previous page in this assignment titled *Use N-Gram Tagging*, create four new variables:

- default_tagger to represent the default POS tag to be used when a tag cannot be identified by another tagger. This value may be 'NN'.
- tagger1 to represent a UnigramTagger that will use default_tagger as its backup tagger if it cannot identify a tag.
- tagger2 to represent a BigramTagger that will use tagger1 as its backup tagger if it cannot identify a tag.
- final_tagger to represent a TrigramTagger that will use tagger2 as its backup tagger if it cannot identify a tag.
- Print tagger1, tagger2, and final_tagger.

Select the TRY IT button when you are ready to run your code.

Your output should display as shown below.

```
<UnigramTagger: size=6257> <BigramTagger: size=1202>
<TrigramTagger: size=518>
```

▼ Solution

The solution is shown below.

```
default_tagger = DefaultTagger('NN')
tagger1 = UnigramTagger(training_tagged_sents, backoff =
    default_tagger)
tagger2 = BigramTagger(training_tagged_sents, backoff = tagger1)
final_tagger = TrigramTagger(training_tagged_sents, backoff =
    tagger2)
print(tagger1, tagger2, final_tagger)
```

Store Taggers

It is more efficient to save already-trained taggers, as training taggers on large corpora takes time.

- Using the Python Module `pickle`, we can save `final_tagger`.
- Using `pickle`, we can serialize objects to files on disk or buffer and also deserialize them back into our Python programs when run.

Variable output is defined by using the `open` function to write to file

`'final_tagger.pkl'`.

- The Python module `pickle` creates files using the file extension `.pkl`.
- `'wb'` stands for 'write binary' when writing the pickled data into the `.pkl` file.

```
output = open('final_tagger.pkl', 'wb')
```

Next, we **serialize** the data.

- Using the `dump` method from the `pickle` module, we convert `final_tagger`'s object hierarchy into a byte stream using `output`.
- Then, we use the `close` method to finish writing to and close the file.

```
dump(final_tagger, output, -1)
output.close()
```

From here, we use the `open` method to open the pickled byte stream from file `'final_tagger.pkl'` using `'rb'` or 'read mode', and store the binary file in `input`.

To save the tagger, we create a new variable called `saved_tagger`, and use the `load` method to read the pickled byte stream.

- To finish editing and close `input`, we use the `close` function.


```
input = open('final_tagger.pkl', 'rb')
saved_tagger = load(input)
input.close()
```

Run your code using the TRY IT button below. If there is no output, then you did everything right. If you get an error message, then go back and check your code.

challenge

Try this variation:

Using the tagger called `saved_tagger` that we just created, tag the POS in the string `test_sent`.

Enter the below code, then select the TRY IT button to run the file.

```
test_sent = "The gruff old man sat in the back of the bait
shop grumbling to himself as he scooped out a
handful of worms."
test_sent_tokens = test_sent.split()

print(saved_tagger.tag(test_sent_tokens))
```

Your output should display as shown below.

```
[('The', 'AT'), ('gruff', 'NN'), ('old', 'JJ'), ('man',
'NN'), ('sat', 'VBD'), ('in', 'IN'), ('the', 'AT'), ('back',
'RB'), ('of', 'IN'), ('the', 'AT'), ('bait', 'NN'), ('shop',
'VB'), ('grumbling', 'NN'), ('to', 'IN'), ('himself',
'PPL'), ('as', 'CS'), ('he', 'PPS'), ('scooped', 'NN'),
('out', 'IN'), ('a', 'AT'), ('handful', 'NN'), ('of', 'IN'),
('worms.', 'NN')]
```

Analyze Taggers

The Brown Corpus has the `tagged_words()` method that return a list of tuples. The elements are the word and its tag. We are going to compare the output of our `final_tagger` and Brown's list of tagged words.

We are going to start by declaring a few variables. Codio will timeout if we try to tag the entire Brown Corpus. Instead `length` represents how many words from the corpus will be used for our comparison. We will start with

the first 1,000 words. The variable `words` are the first 1,000 words from the Brown Corpus. `expected_tags` is list of the first 1,000 words tagged by Brown. Finally, `test_tags` are the first 1,000 words with our tags.

```
length = 1000
words = brown.words()[0:length]
expected_tags = brown.tagged_words()[0:length]
test_tags = final_tagger.tag(words)
```

We are going to iterate over the list of tuples and compare each element in the tuple. If they are different, we are going to increment the variable `different`. Once the comparisons are done, transform `different` into a percentage of how accurate our tagger is.

```
different = 0
for i in range(length):
    check = all(expected == actual for expected, actual in
                zip(expected_tags[i], test_tags[i]))
    if not check:
        different += 1

print(f'{100 * (length - different) / length}%')
```

Select the TRY IT button to run the file. Your output should display as shown below.

```
76.3%
```

challenge

Try this variation:

Increase the number of words used for our analysis. Run the program after each change.

```
length = 10000
```

```
length = 50000
```

While the accuracy score is not exactly the same, it is close to 76%.

Note, if you increase the value of `length` too much, Codio will time out. Enter the below code, then select the TRY IT button to run the file.

If you want to see the differences between the two taggers, change your loop to look like the following:

```
for i in range(length):
    check = all(a == b for a, b in zip(expected_tags[i],
                                       test_tags[i]))
    if not check:
        print(f'Expected: {expected_tags[i]}')
        print(f'Actual: {test_tags[i]}')
        print(30 * '-')
```

Formative Assessment 2

Assume the following code:

```
import nltk
nltk.download('universal_tagset')
from nltk import FreqDist
from nltk.corpus import brown

brown_news_tagged = brown.tagged_words(categories='news',
                                       tagset='universal')
freq_dist_tags = FreqDist(tag for (word, tag) in
                          brown_news_tagged)
print(freq_dist_tags.most_common())
```